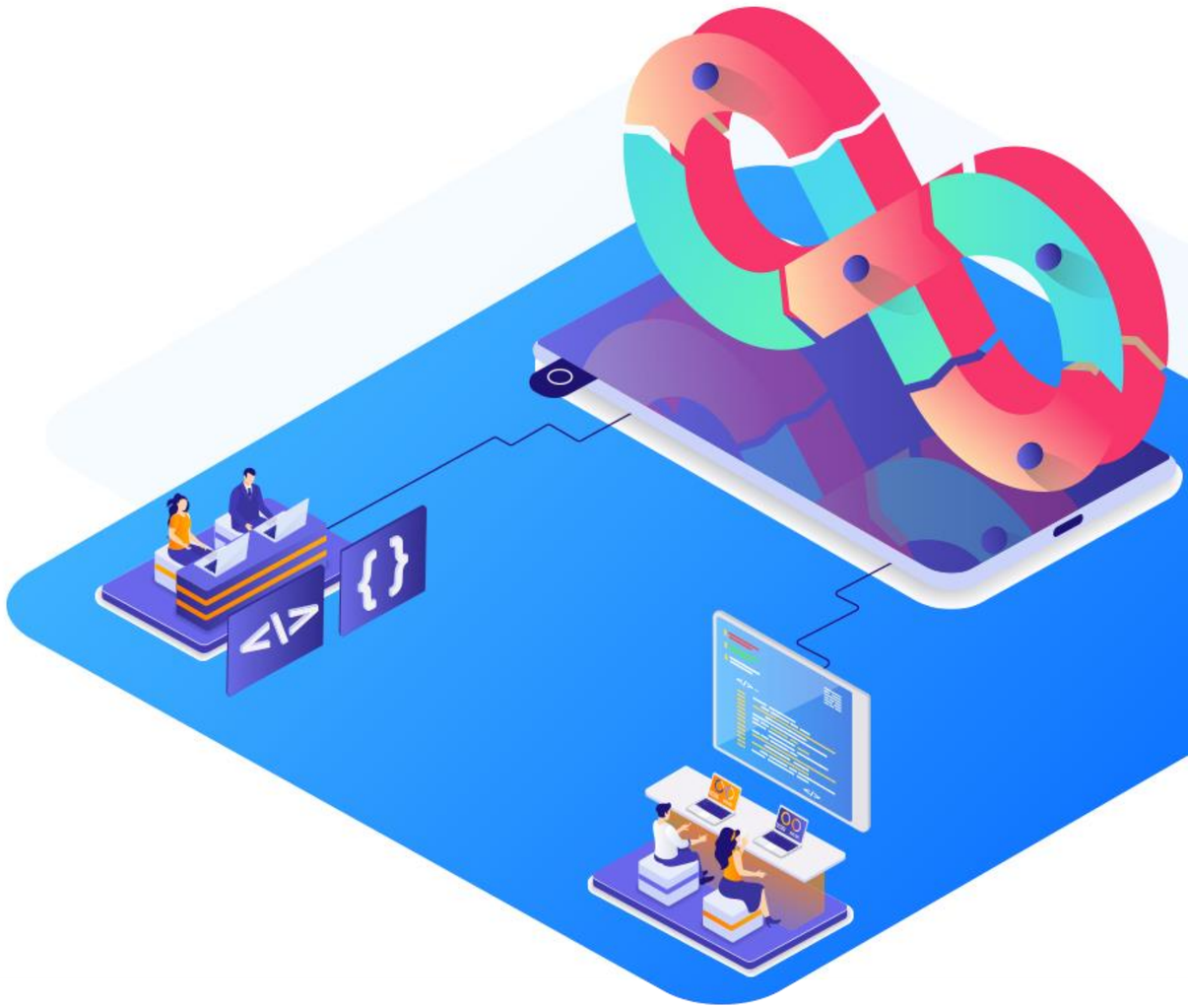
DevOps on AWS



Continuous Integration and Continuous Delivery (CI/CD) on AWS

Learning Objectives

By the end of this lesson, you will be able to:

- Create a pipeline from a CodeCommit repository to deploy applications automatically upon code changes
- Implement triggers at various stages of the pipeline for automated build, test, and deployment processes
- Build an application using CodeBuild and deploy it on the S3 bucket to ensure consistent and repeatable builds and deployments
- Set up an AWS Elastic Beanstalk environment to deploy an application for easy scaling and management of applications
- Analyze and differentiate various AWS deployment strategies to optimize performance, cost, and availability

Understanding the CI/CD pipeline

Overview of the CI/CD Pipeline

Continuous integration: It automates code integration, enabling frequent collaboration and early issue detection.

Continuous deployment: It automates code delivery to production, ensuring rapid and reliable releases.

CI/CD pipeline: It automates the end-to-end software development process, fostering speed, quality, and collaboration.

Features of CI/CD Pipeline

01

Automation:
Compile automatically, build, and test code

02

Version control:
Manage code changes using version control systems

03

Continuous integration:
Integrate code frequently for collaboration and early issue detection

04

Continuous development:
Deliver code rapidly and automatically to production environments

Features of CI/CD Pipeline

05

Deployment orchestration:
Control and stage deployment processes

06

Environment parity:
Maintain consistent environments from development to production

07

Rollback mechanisms:
Implement safeguards for handling deployment failures

08

Security integration:
Integrate security checks throughout the pipeline for enhanced protection

Quick Check

Your development team frequently faces issues with delayed releases due to manual testing and deployment processes. To overcome this, your team plans to implement an automated system that integrates code changes more frequently and deploys them seamlessly. What set of practices should your team adopt to achieve this goal?

A. Continuous Integration/Continuous Deployment

B. Continuous Innovation/Continuous Deployment

C. Central Integration/Continuous Deployment

D. Complete Integration/Complete Deployment

AWS CodePipeline: Overview and Setup

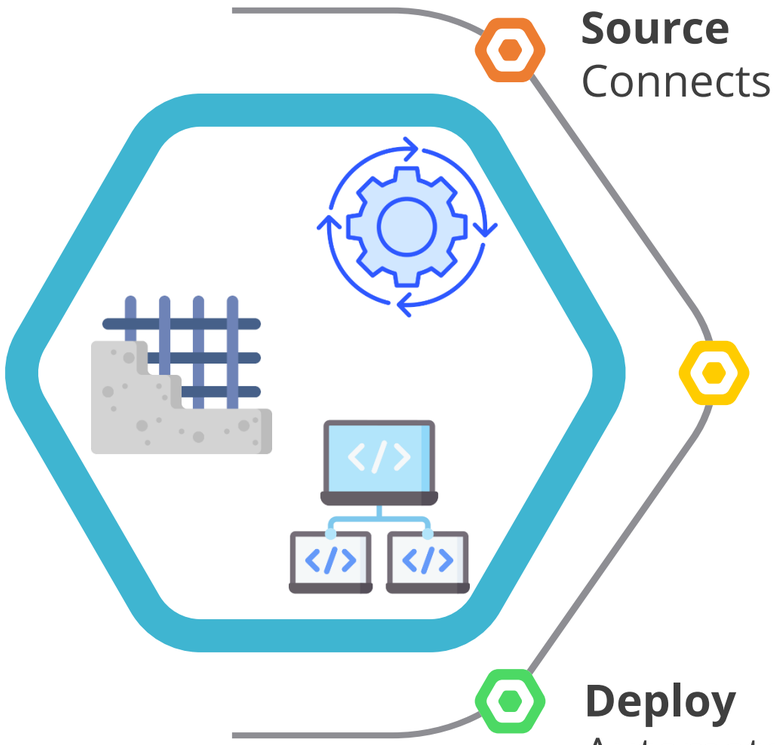
What Is AWS CodePipeline?

It is a fully managed service for modeling, visualizing, and automating software release steps with speed and reliability in consideration.

It streamlines and automates CI/CD pipeline components for seamless software delivery.

Key Components

AWS CodePipeline is built upon three fundamental components that orchestrate the software delivery process seamlessly, which are as follows:



Source
Connects to the code repository

Build
Compiles and packages source code

Deploy
Automates deployment

Why AWS CodePipeline?

It offers a robust set of benefits that enhance software delivery processes, such as:

01

Efficiency:
Streamline software delivery and reduce errors

02

Speed:
Accelerate code deployment

03

Consistency:
Guarantee dependable software releases

04

Versatility:
Adapt to diverse project needs

Why AWS CodePipeline?

It offers a robust set of benefits that enhance software delivery processes, such as:

05

Collaboration:
Promote team cooperation with the central code integration platform

06

Scalability:
Expand delivery pipelines for varying project scopes

07

Compliance:
Ensure regulatory adherence with automated checks

08

Cost-effective:
Minimize manual deployment expenses

Where to Use AWS CodePipeline?

Its adaptability spans various environments and integrates with tools to streamline software delivery in diverse scenarios, such as the following:

01

AWS Cloud:
Is ideal for deploying on AWS

02

Hybrid Environments:
Is compatible with hybrid cloud setups

03

Third-Party Ecosystem:
Is adaptable to various third-party services

AWS CodePipeline: Integrating with Other AWS Services

AWS CodePipeline seamlessly integrates with a wide range of AWS services, enhancing flexibility and functionality.

Examples of integration:

AWS Elastic Beanstalk:

Orchestrates deployments

AWS CodeBuild:

Manages testing and building source code

Advantages of Integration

Here are some of the advantages of integrating AWS CodePipeline with other services:

Serverless integration:

- Integrates serverless computing platforms like AWS Lambda into the pipeline for serverless deployments
- Empowers tailored workflows for specific requirements

Efficiency and automation:

- Automates and orchestrates different stages of the software delivery process
- Reduces manual intervention by streamlining development and deployment

How to Use AWS CodePipeline?

The following is the flow on how to use AWS CodePipeline:

1. **Integration:** Integrate AWS CodePipeline into the CI/CD workflow by defining pipeline stages, actions, and source code repositories
2. **Custom workflows:** Customize the CI/CD workflows to match the project's requirements by incorporating pre- and post-build actions and scripts
3. **Trigger automation:** Automate the entire software release process with AWS CodePipeline
4. **Monitoring:** Observe the CI/CD pipeline's progress, build logs, and results

Assisted Practice

Creating a Pipeline from the GitHub Repository **Duration: 15 Min.**

Problem Statement:

You have been assigned a task to set up an AWS CodePipeline for continuous integration and delivery (CI/CD) using a GitHub repository.

Outcome:

You can create an AWS CodePipeline for CI/CD using a GitHub repository and configure its stages.

Note: Refer to the demo document for detailed steps:
01_Creating_a_Pipeline_from_the_GitHub_Repository

Assisted Practice: Guidelines

Steps to be followed:

1. Create a new pipeline using the GitHub repository

ASSISTED PRACTICE

What Are Triggers?

It automates processes, enables event-driven architectures, and streamlines the execution of various AWS services and resources.

Types of triggers:

01

Manual triggers

02

Automated triggers

Triggers act as critical control points within AWS CodePipeline, enabling precise management of the software delivery process.

Manual vs. Automated Triggers

Manual triggers

- Human intervention: Manual triggers require human approval or intervention before proceeding.
- Quality assurance: They serve as checkpoints for careful review and verification of changes.
- Common use cases: Compliance checks, approvals, and critical change reviews

Automated triggers

- Event-driven: Automated triggers respond automatically to specific events or code changes.
- CI/CD backbone: They are the foundation of continuous integration and continuous deployment.
- Examples: Code repository updates triggering builds, tests, and deployments without intervention

AWS CodeBuild: Overview and Setup

What Is AWS CodeBuild?

It is a fully managed continuous integration service that compiles source code, runs tests, and produces ready-to-deploy software packages.

It is a crucial component in the CI/CD pipeline for automating build and testing processes.

Why Choose AWS CodeBuild?

It automates, scales, integrates, and guarantees reliable results for efficient DevOps workflows and accelerated software delivery. Following are some of its advantages:

01

Focused build and test: Provide a dedicated service for automating build and test processes

02

Build artifact ggeneration: Create versatile artifacts for your CI/CD pipeline

03

Integration: Integrate seamlessly with AWS services

Why Choose AWS CodeBuild?

03

Reliability:
Ensure consistent, reliable, and reproducible builds

04

Flexibility:
Scale resources as needed, accommodating projects of all sizes

05

Efficiency:
Automate build processes, reducing manual effort

Where to Use AWS CodeBuild?

It can be useful in the following scenarios:

01

AWS cloud:
Is ideal for building applications and services on AWS

02

Multi-cloud deployments:
Works in multi-cloud setups

03

Integration with DevOps tools:
Is a part of broader DevOps CI/CD setups

How to Use AWS CodeBuild?

Below is a flow on how to use AWS CodeBuild:

1. **Create build pprojects:** Define build projects with build specifications and source code locations
2. **Source integration:** Connect CodeBuild to the source code repository (e.g., AWS CodeCommit, GitHub)
3. **Customization:** Configure build environments, pre- and post-build steps, and notifications
4. **Trigger automation:** Automate builds with code commits or integrate with AWS CodePipeline for end-to-end automation

Assisted Practice

Building and Deploying a React Application with AWS CodeBuild and S3 **Duration: 25 Min.**

Problem Statement:

You have been assigned a task to automate the continuous integration process as part of a DevOps implementation by implementing a CodePipeline to perform build automation for React source code and deploying it to an S3 bucket.

Outcome:

You will be able to automate the build and deployment of a React application using AWS CodeBuild and deploy it to an S3 bucket using AWS CodePipeline.

Note: Refer to the demo document for detailed steps:
02_Building_and_Deploying_a_React_Application_with_AWS_CodeBuild_and_S3

Assisted Practice: Guidelines

Steps to be followed:

1. Create a React GitHub repository
2. Create an S3 bucket
3. Configure CodeBuild and CodePipeline to perform build and test automation

ASSISTED PRACTICE

Secure CI/CD Practices with AWS

Introduction to Secure CI/CD

Security is a fundamental aspect of software delivery. The importance of security in a CI/CD pipeline is paramount for several critical reasons:

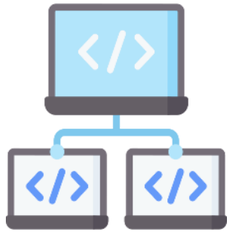
Protection of Assets and Reputation: Security shields against financial losses, legal consequences, and reputational damage from breaches.

Compliance and Data Privacy: Security protects assets, ensures compliance, and provides a competitive edge against evolving cyber threats.

Mitigating Cyber Threats: Security ensures early detection of threats and protection against supply chain vulnerabilities.

Implementing Security in CI/CD Pipelines

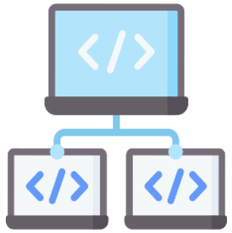
Below are the key security practices and tools:



- **Security as code:** Ensure automated and version-controlled security
- **Shift left security :** Start security checks early
- **Automated security scanning:** Integrate SAST and DAST for code and app security

Implementing Security in CI/CD Pipelines

Below are the key security practices and tools:



- **Dependency scanning:** Check third-party components
- **Container security:** Secure container images with scanning tools like Clair or Trivy
- **Security gates and policies:** Provide policies for a secure code deployment

Securing CI/CD Pipeline Using IAM

Securing a CI/CD pipeline with IAM involves multiple steps, such as:

1. Establish a dedicated IAM user for CI/CD purposes and grant access
2. Furnish the IAM user's credentials to the CI/CD system
3. Implement automated processes for periodically rotating and revoking AWS credentials
4. Embrace the principle of least privilege, ensuring users are granted necessary permissions
5. Eliminate the use of hard-coded secrets from Jenkins files and related CI configurations

Secrets Management with AWS Services

It involves securely storing, retrieving, and managing sensitive information such as passwords, API keys, and encryption keys.

Secrets Manager improves security posture by eliminating the need to hard-code the credentials in the application's source code.

Deployment

Deployment vs. Continuous Deployment

Below are the fundamental differences between deployment and continuous deployment:

Deployment

- Deployment is releasing and installing software into production.
- It is a manual or semi-automated process.
- It requires human intervention and review.
- It may not always have an immediate rollback.

Continuous Deployment

- Continuous Deployment auto-deploys tested code to production.
- It is a fully automated deployment process.
- It reduces human intervention.
- It includes immediate rollback capabilities.

Deployment vs. Release

Software deployment

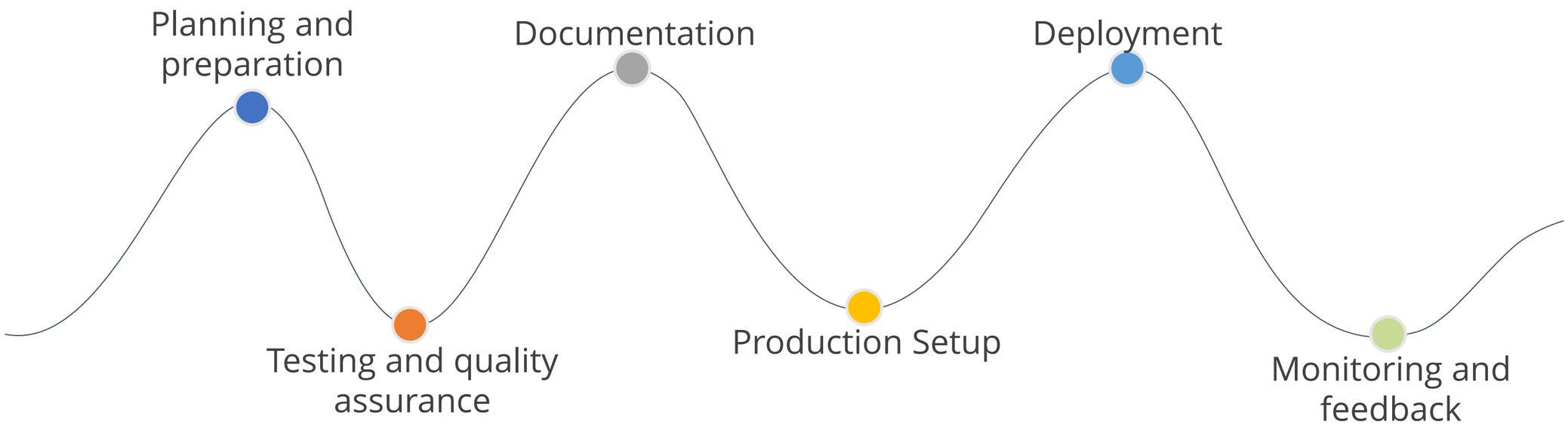
- It is the process of installing software in each environment.
- It focuses on the technical aspects of getting the software up and running in a target environment.
- It is often internal, such as for IT or DevOps teams.
- It ensures software functions in the target environment.

Software release

- It is the formal distribution of a version of software to users.
- It comprises the entire process of preparing the software for distribution.
- It is for the end-users or customers.
- It provides value, updates, or fixes to users.

Transition from Development to Production

The transition from development to production involves the shift from designing and building to making the product ready for use. Here are the key steps involved in this transition:



Introduction to AWS Elastic Beanstalk

It is a Platform-as-a-Service (PaaS) used for software deployment. It simplifies the deployment and scaling of web applications and services.

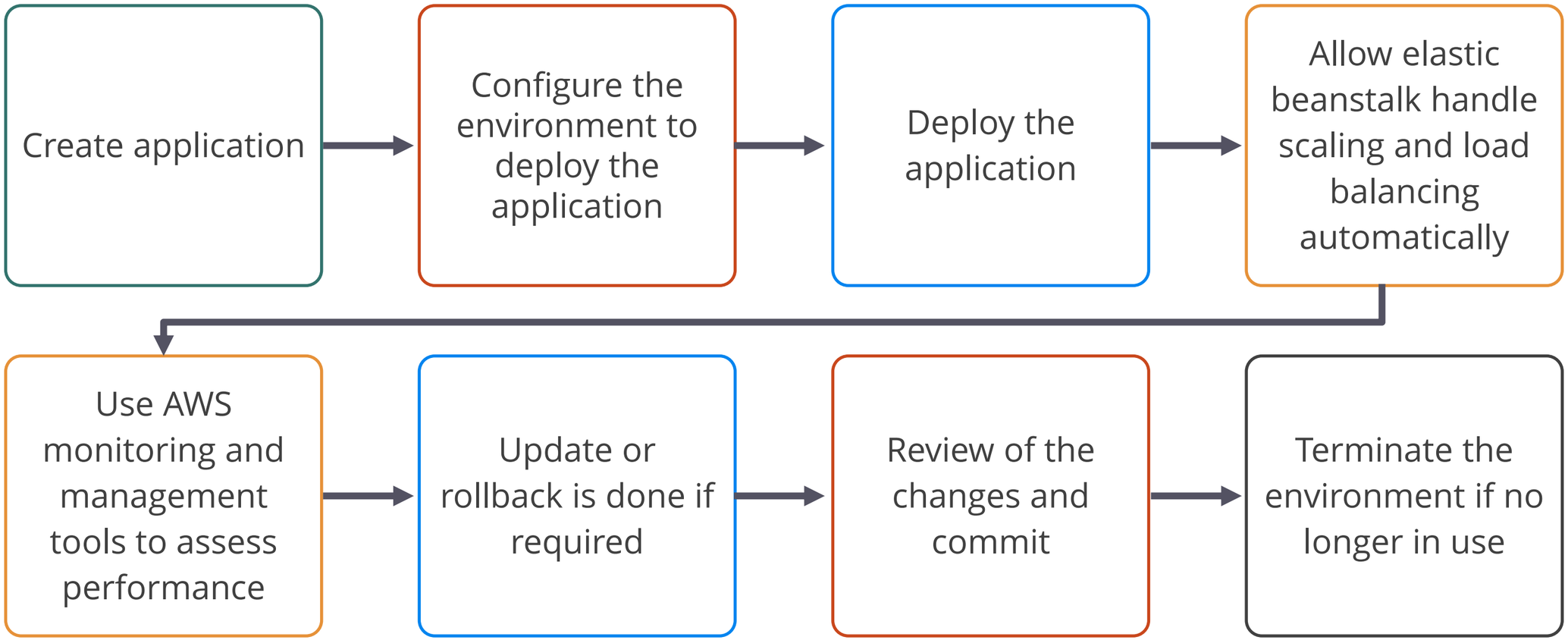
It is designed to let developers and organizations deploy web applications, APIs, and microservices without delving into the underlying infrastructure details.

AWS Elastic Beanstalk: Key Features

- Ease of development
- Auto-scaling
- Managed environment
- Infrastructure customization

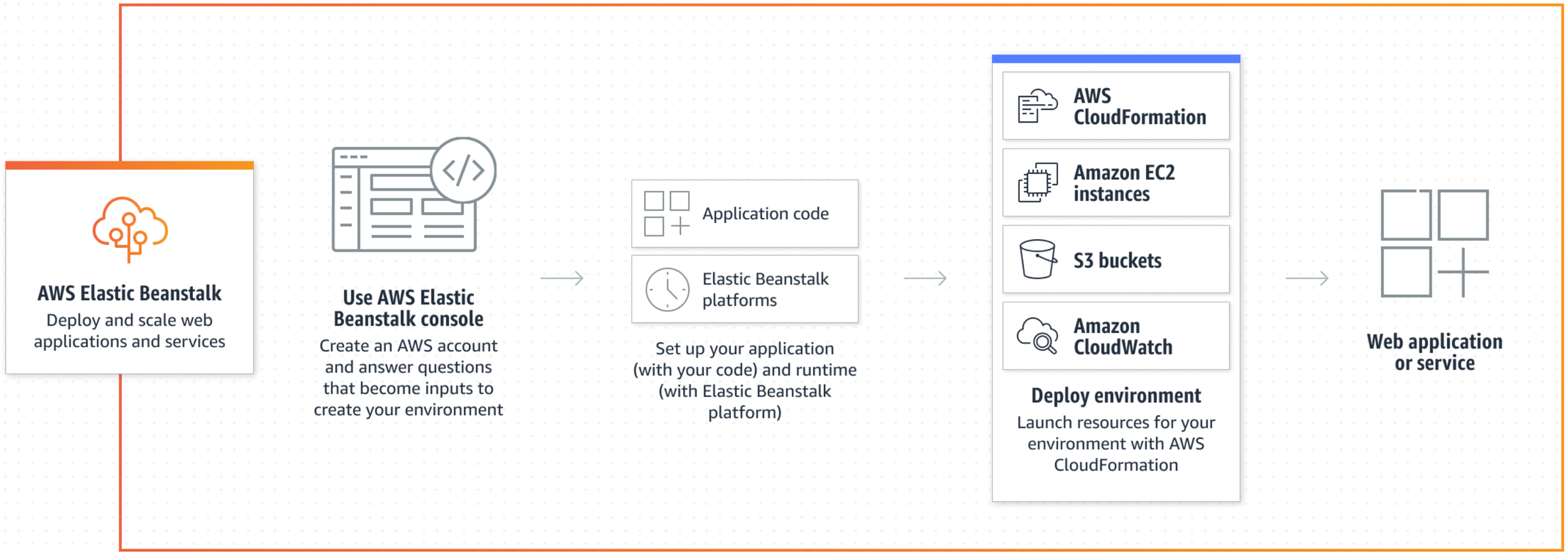
- Multi-language support
- Database integration
- No additional charges
- Monitoring and analytics

AWS Elastic Beanstalk: Workflow



AWS Elastic Beanstalk: Workflow

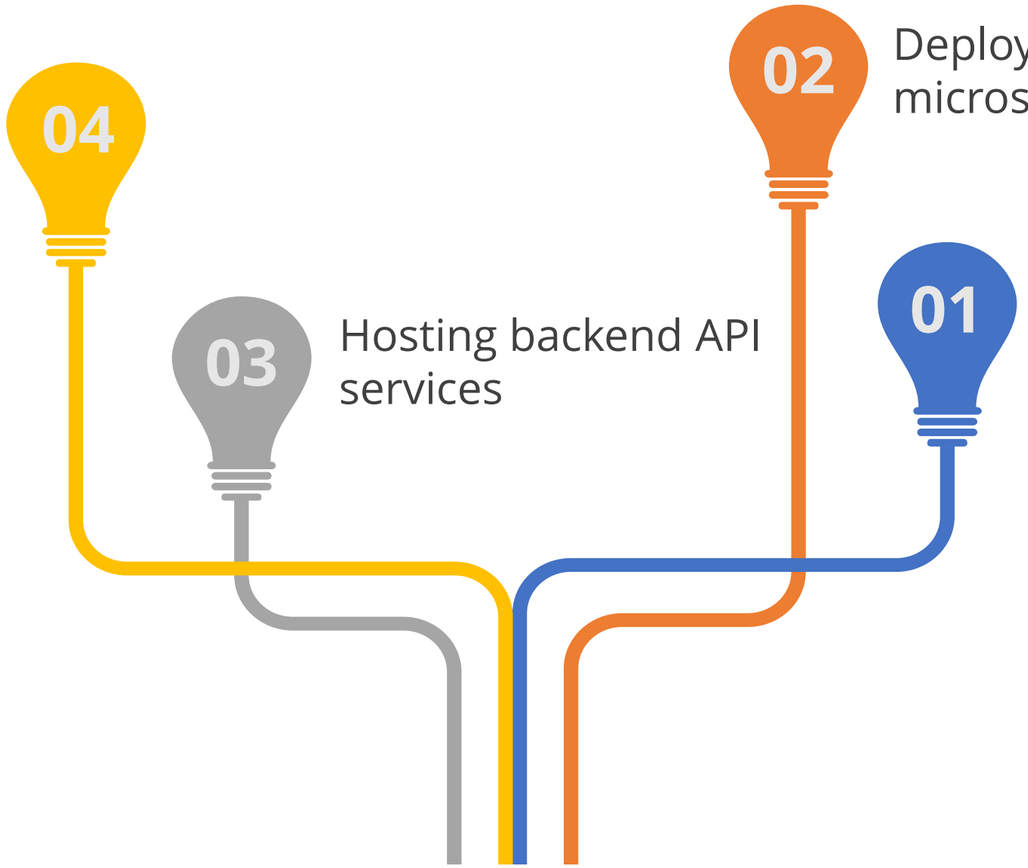
The following diagram shows the seamless workflow of AWS Elastic Beanstalk:



Source:https://aws.amazon.com/elasticbeanstalk/

AWS Elastic Beanstalk: Use Cases

Elastic Beanstalk simplifies the deployment and management of web applications. Its use cases include:



01 Hosting web application

02 Deploying and scaling microservices

03 Hosting backend API services

04 Hosting content management system platforms

Companies Using Elastic Beanstalk

There are around 11,200 organizations using AWS Elastic Beanstalk. Among them most of the organizations are US-based, a few of them are listed below:

Accenture

powercloud

Kellogg's

GE Oil and Gas

United Airlines

Primer

Glovo

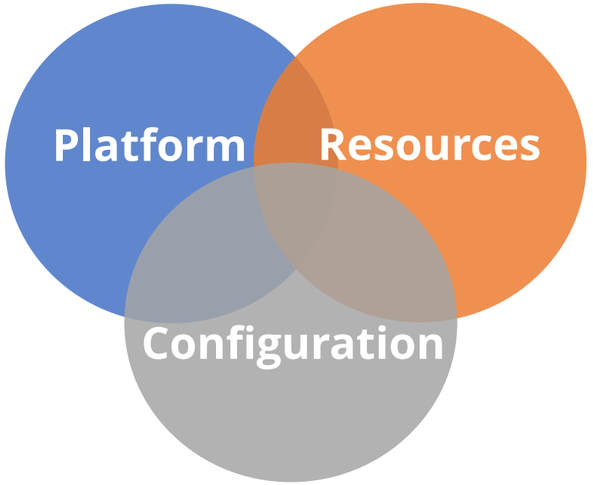
OYOROOMS

AWS Elastic Beanstalk Environment

An environment in AWS Elastic Beanstalk is a collection of AWS resources that provide infrastructure to deploy, manage, and scale web apps and services.

The components of the Beanstalk environment are:

OS, Runtime, App server, Software stack



EC2 instances, Load balancers, Databases

Resource types, Scaling rules, Software settings

AWS Elastic Beanstalk: Environment Features

It is a container that holds everything necessary to run an application. The following are the features of the Beanstalk environment:

Management
Create, modify, and monitor application deployment seamlessly

Runtime configuration
Configure various settings such as platform version, environment variables, resource allocation, etc

Scaling
Scale the application easily by adding instances to handle the load

Deployment
Deploy different versions of your application to multiple environments simultaneously

Types of Beanstalk Environment

AWS Elastic Beanstalk provides two primary types of environments, each designed for specific use cases:

Web server environment

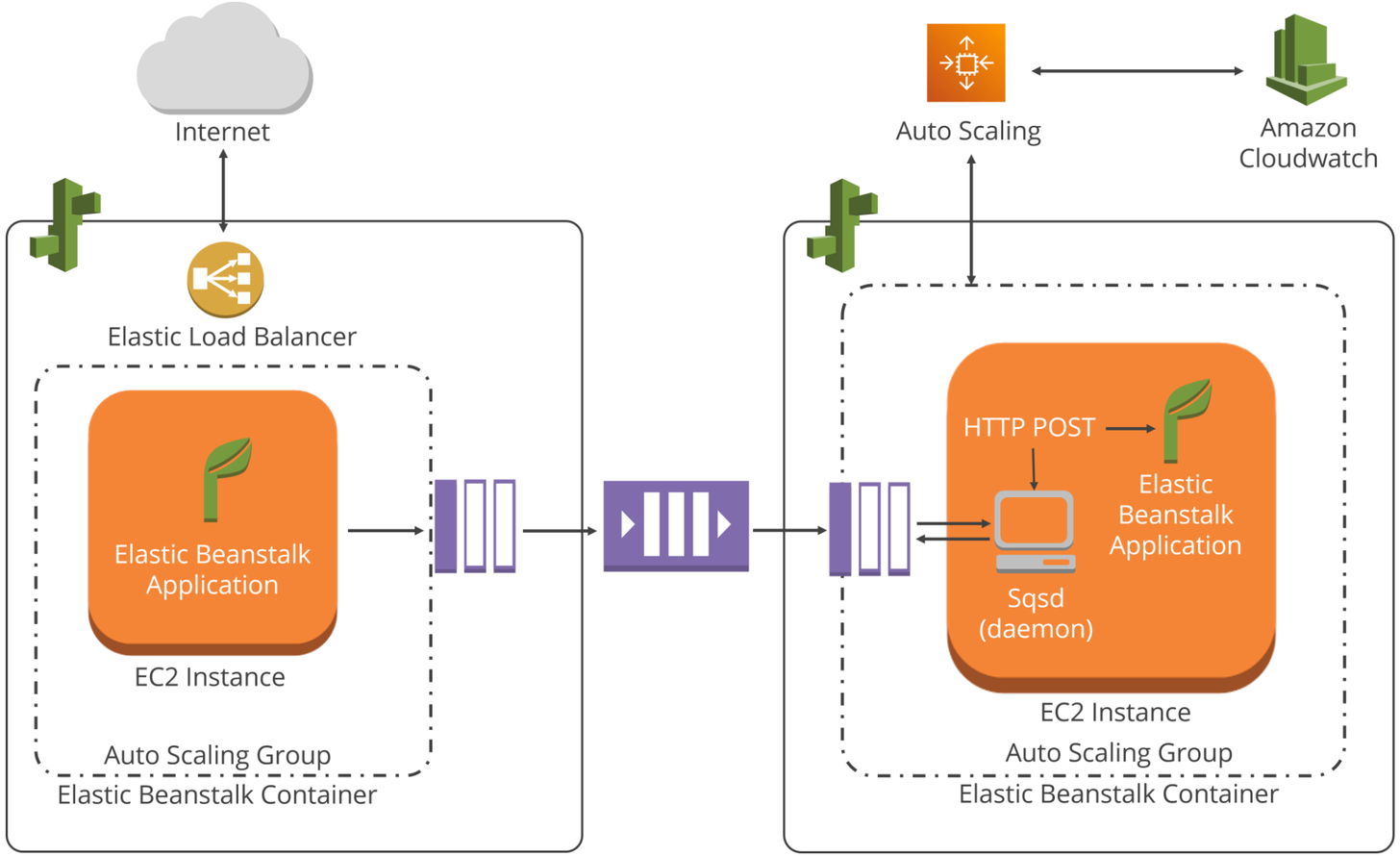
- Serves web apps and services through HTTP/HTTPS
- Utilizes Elastic Load Balancer (ELB), Amazon EC2 instances, and an optional relational database
- Is ideal for web UIs and RESTful APIs
- Auto-scales with traffic

Worker environment

- Processes background long-running tasks without HTTP/HTTPS
- Utilizes Amazon SQS and EC2 for processing
- Is ideal for long-running or scheduled tasks
- Scales based on queue depth or metrics

Working of Beanstalk Environment

The diagram below depicts component interactions within the Beanstalk environments and AWS services:



Source: https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/concepts-worker.html

Supported Platforms on Elastic Beanstalk

In AWS Elastic Beanstalk, a platform denotes the integrated environment consisting of the operating system, runtime, application server, and software stack required to execute the application.

Following are the platforms supported by AWS Elastic Beanstalk:

- Docker
- Go
- Java SE
- Tomcat
- .NET Core on Linux

- .NET on Windows Server
- Node.js
- PHP
- Python
- Ruby

Beanstalk Environment Resources

In Elastic Beanstalk, several resources are provided to ensure the smooth operation of an application. These resources are:

- Amazon EC2 virtual machine
- EC2 security group
- Elastic Load Balancer
- Amazon Relational Database Service (RDS)

- Amazon S3 bucket
- Amazon CloudWatch alarms
- AWS CloudFormation stack
- Amazon Simple Notification Service (SNS)

Beanstalk Environment Configurations

It refers to the set of parameters and settings that determine how the environment and its resources must work.

The following are the configurations of a Beanstalk environment:

Platform version

It defines the software stack (OS, runtime, app server).

Instance type

It specifies the size and capability of the EC2 instances (e.g., t2.micro, m5.large).

Beanstalk Environment Configurations

Scaling

It configures auto-scaling rules, such as the number of instances, scaling triggers, etc.

Load balancer

It defines the load balancer type and related settings.

Database

It defines the Relational Database Service (RDS) configurations.

Environment variables

It provides the key-value pairs for the app settings.

Beanstalk Environment Configurations

Logging

It specifies the settings for storage and retention of logs.

Notifications

It configures alerts and notifications for environment events.

Network

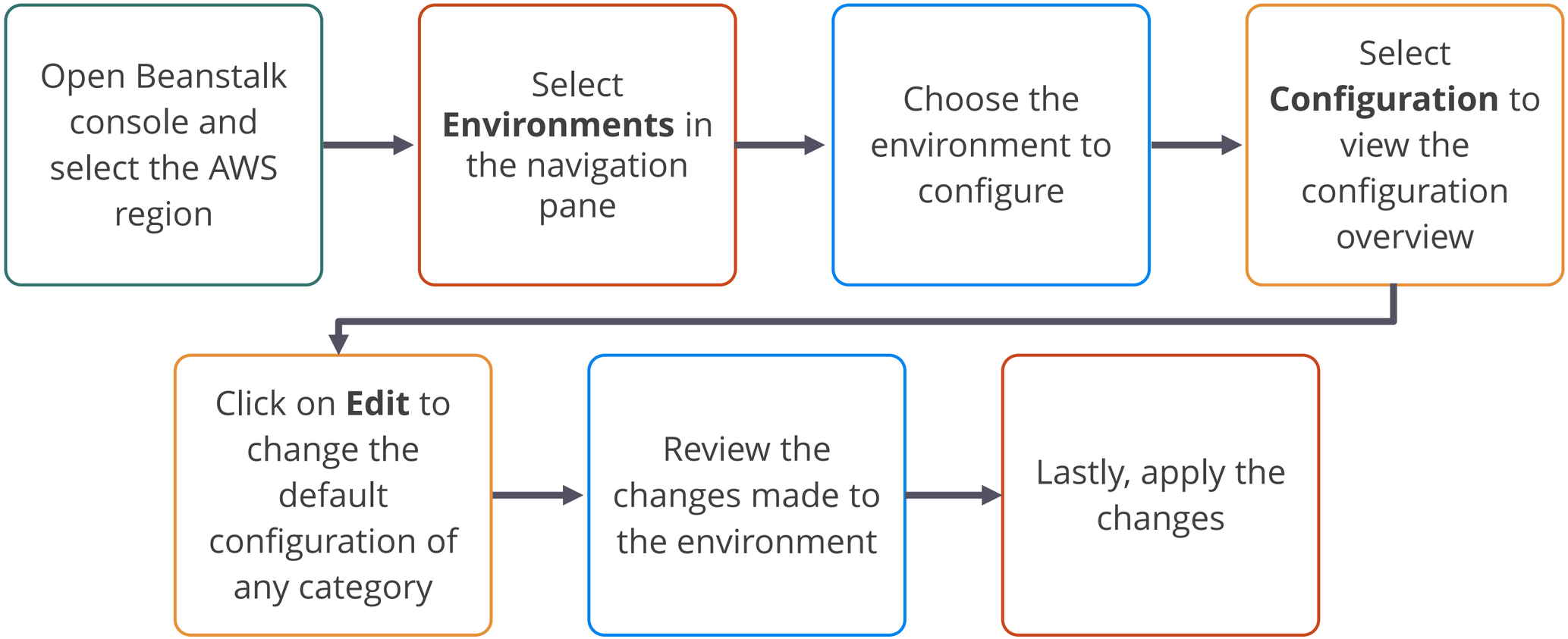
It defines VPC, subnets, security groups, and other networking settings.

Software

It customizes software settings, packages, and extensions specific to the platform.

Configuring Beanstalk Environment

Below are the steps for environment configuration using the Elastic Beanstalk console:



Launching an Application on Beanstalk

Below are the steps to launch an application on AWS Elastic Beanstalk:

1. Navigate to the AWS Management Console and sign in with your AWS account credentials

2. Search for Elastic Beanstalk in search bar

3. Click on **Get Started** or **Create New Application**

Launching an Application on Beanstalk

4. Provide the appropriate name and description for the application

5. Provide a **Tag** which is typically a key-value pair

6. Click on **Create** to create an application

Launching an Application on Beanstalk

7. Click on **Actions > Create environment** within the application page

8. Select the type of environment required to launch the application

9. Specify the environment details like name, description, domain

Launching an Application on Beanstalk

10 Configure the **Base Configuration** of the environment by specifying the type of platform and application code

11 Customize the environment by clicking on **Configure more options**

12 Click on **Create environment a**fter configuring the environment

Assisted Practice

Setting up Java-Based Elastic Beanstalk Environment **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to automate Platform as a Service (PaaS) deployment using AWS Elastic Beanstalk without creating infrastructure components or hosting supported programming language artifacts.

Outcome:

You can set up and deploy a Java-based application using AWS Elastic Beanstalk, automating the Platform as a Service (PaaS) deployment process.

Note: Refer to the demo document for detailed steps:
03_Setting_up_Java_Based_Elastic_Beanstalk_Environment

Assisted Practice: Guidelines

Steps to be followed:

1. Create an IAM role for AWS Elastic Beanstalk
2. Create a Java-based Elastic Beanstalk application and environment

ASSISTED PRACTICE

Quick Check

Your team needs to deploy an application with EC2 instances, S3 buckets, and IAM roles using AWS CloudFormation. To handle frequent updates, which CloudFormation feature helps you manage changes in a scalable and repeatable way?

A. Automated security scanning

B. Container orchestration

C. Application deployment and scaling

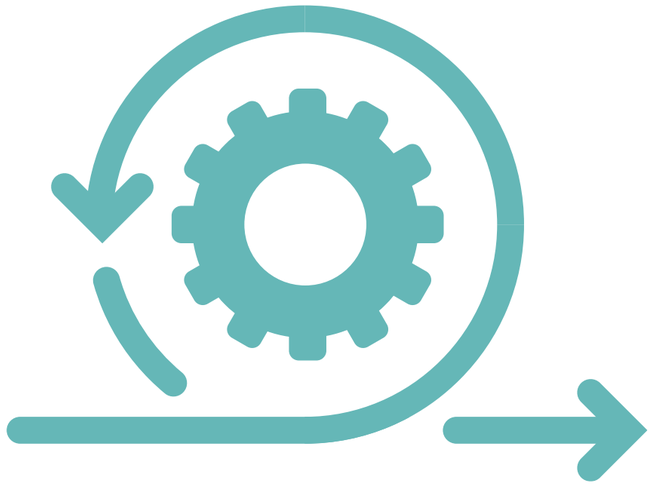
D. Real-time log analysis

Deployment Strategies on AWS

Deployment Strategies

It refer to the methods used to manage the release of new features, updates, or versions of an application into a runtime environment.

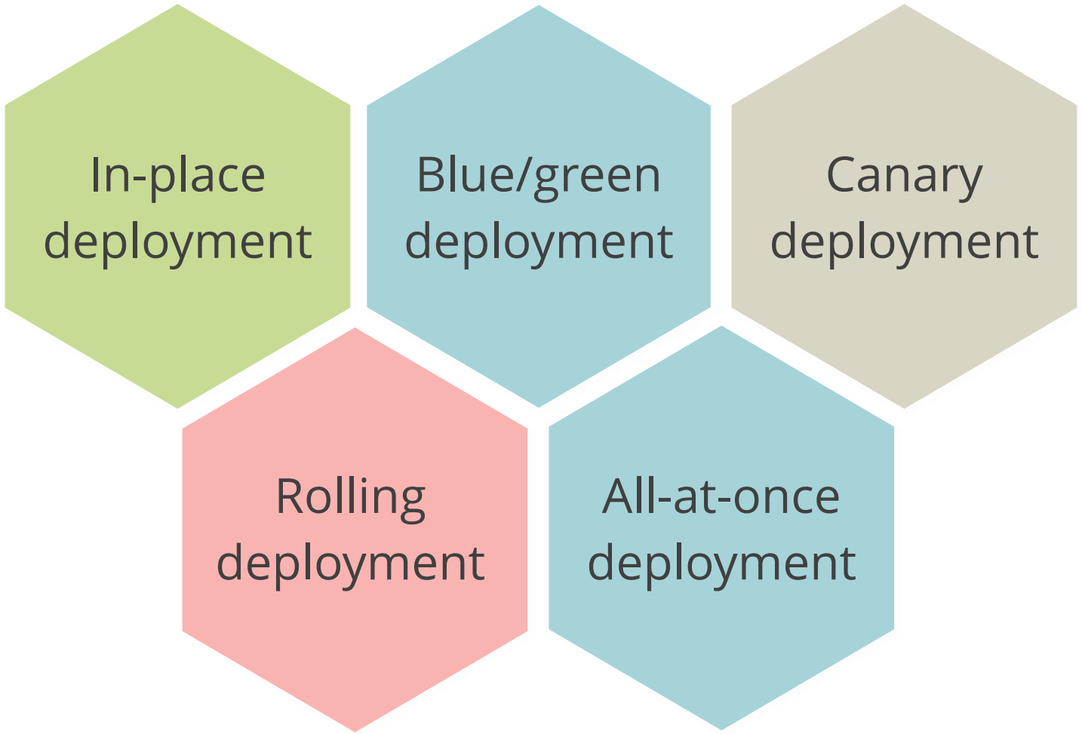
The following are ways in which deployment strategies enhance the development process:



- Minimize downtime
- Improve product quality
- Efficient resource utilization

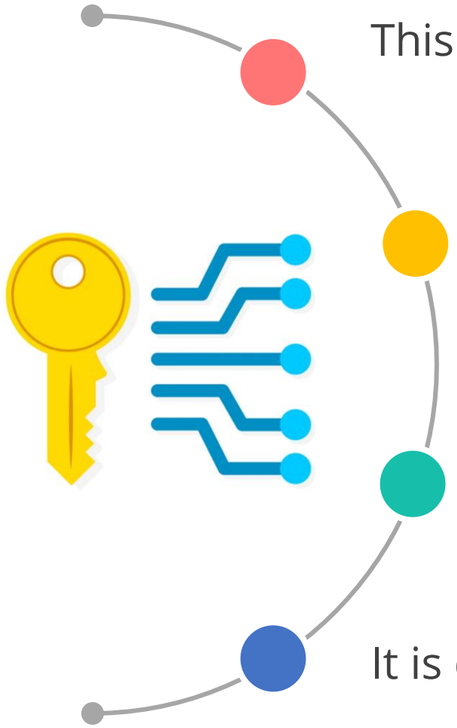
AWS Deployment Strategies

The following are the general-purpose deployment strategies that can be used with various AWS services:



In-place Deployment Strategy

It involves updates or changes to a software application or system directly in its current environment, without creating entirely new resources.



This strategy maintains and improves software systems while they are in production.

It involves techniques to ensure gradual and controlled changes to the system.

It is a suitable approach to minimize downtime.

It is cheaper than other strategies as no new infrastructure is provisioned.

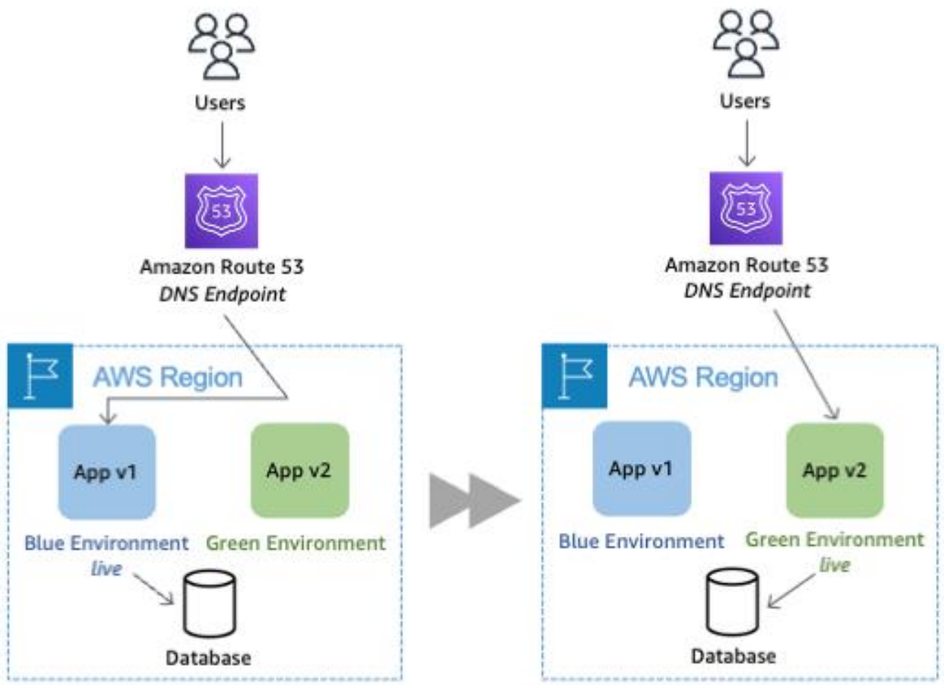
In-place Deployment Strategy

The following depicts the workflow involved while using the in-place deployment strategy:

- Prepare the environment by ensuring it is ready and necessary backups are taken
- Update the current application to the new version directly on the existing infrastructure
- Handle the short downtime or reduced performance during the process of deploying updates
- Validate that the new version functions correctly and integrates seamlessly with other components

Blue/Green Deployment Strategy

This strategy involves switching between two identical environments, one of which serves as the production environment and the other as the staging or testing environment.



Source: https://docs.aws.amazon.com/whitepapers/latest/blue-green-deployments/introduction.html

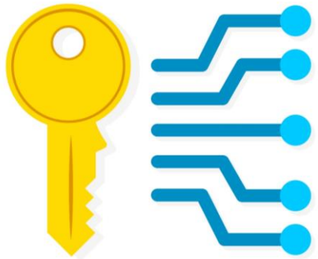
Blue/Green Deployment Strategy

The following depicts the workflow involved while using the Blue/Green deployment strategy:

- The current production environment (blue) continues to run the live application.
- The new version of the application is deployed and tested in the staging environment (green) without impacting users.
- Once the testing is successful, a router or load balancer is used to switch user traffic from the blue environment to the green one.
- If any issues arise in the green environment, traffic can quickly be reverted to the blue environment, ensuring minimal disruption.

Blue/Green Deployment Strategy

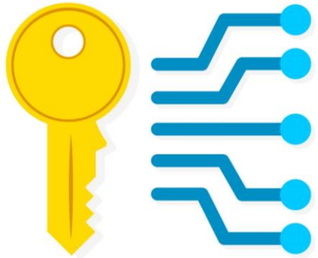
The following are the key points regarding the Blue/Green deployment strategy:



- It allows thorough testing of the new version without impacting users reducing the downtime.
- It significantly reduces the risk of unexpected problems affecting users.
- It encourages a culture of continuous improvement as teams can iterate on the process and refine it over time.
- It ensures a quick rollback and minimal disruption in case any issues are discovered in either of the environments after deployment.

Rolling Deployment Strategy

It involves updating a software application incrementally by replacing instances of the old version with instances of the new version, typically one at a time.



- It ensures that the application remains available throughout the deployment process.
- It maintains high availability by gradually transitioning to the new version while keeping the old version running.
- It can easily rollback to the previous version if issues arise with the new version
- It is commonly used in container orchestration platforms like Kubernetes.

Rolling Deployment Strategy

The following depicts the common workflow involved while using the rolling deployment strategy:

- The new version of the application is ready for deployment, while the old version is still in operation.
- The rolling deployment replaces or upgrades the application on each instance one by one or in subsets.
- During this process, some instances run the old version while others run the new one.
- Once all instances are updated, the rolling deployment is complete.

Canary Deployment Strategy

It is a method of rolling out new software versions in a controlled manner. It works in the following manner:

Progressive rollout
The traffic is split between the existing and new versions of the application.

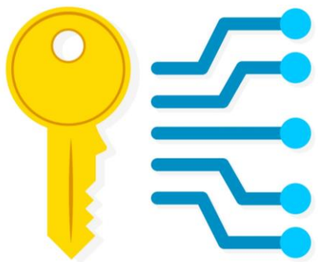
Subset testing
The new version is rolled out to a small subset, allowing real-world testing.

Traffic control
Traffic in large amounts is directed to the new version of the application.

Full rollout
The new version of the application is rolled out to users once stability is ensured.

All-at-once Deployment Strategy

It is a software release method that introduces a new version of an application instantaneously.



- It allows an immediate shift of traffic from the old version of the application to the new one.
- It can provide quick access to the new version for all users at once.
- It is beneficial when the speed of deployments is required.
- It is ideal for deployment in non-live environments like development and test environments.

Blue/Green vs. Rolling Deployment

Blue/Green deployment

- It involves having two identical environments; one is current (blue), and the other is testing (green).
- It deploys the new version in an isolated environment (green).
- It switches the production environment from blue to green when the new application version is thoroughly tested.

Rolling deployment

- It has only one environment; the new code is deployed in a staggered manner.
- It deploys the new version to a small portion of the infrastructure.
- It continues deploying to additional servers or instances in a controlled manner until all are updated with the new application version.

Rolling vs. Canary Deployment

Rolling deployment

- It gradually deploys new application versions to a subset of servers, while the old version remains available.
- It helps minimize downtime and risks by ensuring that at least a portion of the application is always running.
- It is often used for applications where high availability is critical, such as web servers.

Canary deployment

- It involves releasing a new application version to a small group of users or servers before a complete rollout.
- It allows for early testing and validation of the new version's performance.
- It is ideal for experimenting with new features or changes, especially in larger applications.

Assisted Practice

Implementing a Blue-Green Deployment Using CodeDeploy **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to implement a Blue-Green deployment strategy using AWS CodeDeploy and gain proficiency in setting up a continuous integration pipeline using AWS CodePipeline and CodeBuild for automating Maven-based build and test processes.

Outcome:

You can implement Blue-Green deployments using AWS CodeDeploy and automate Maven builds and tests with CodePipeline and CodeBuild.

Note: Refer to the demo document for detailed steps:
04_Implementing_a_Blue_Green_Deployment_Using_CodeDeploy

Assisted Practice: Guidelines

Steps to be followed:

1. Import the repository from GitHub
2. Configure CodeBuild and CodePipeline to perform build and test automation

ASSISTED PRACTICE

Quick Check

Your team is preparing to release a major update to a production application. To avoid downtime and ensure a smooth transition, you plan to deploy the new version in a separate environment and gradually switch over user traffic once the new version is confirmed stable. What deployment strategy should you use to accomplish this?

A. Sequentially replace instances

B. Shift traffic between two environments

C. Test in isolation before going live

D. Deploy to a small subset first

Monitoring and Managing CI/CD Pipelines

Monitoring and Logging using AWS CloudWatch

It enables DevOps teams to collect and track metrics, collect and monitor log files, and set alarms.

AWS CloudWatch is used for:

- **Metric Collection:** CloudWatch can collect and visualize key performance metrics of AWS resources.
- **Log Aggregation:** It enables the centralization of logs from various AWS services and applications.
- **Alarm Creation:** It can set up alarms based on predefined thresholds or custom metrics.
- **Integration with AWS Services:** It seamlessly integrates with other AWS services.

Monitoring Aspects of a CI/CD Pipeline

The essential aspects to monitor for a smooth and effective application deployment using the CI/CD pipeline in AWS CodePipeline are as follows:

01 Source code changes

02 Deployment status

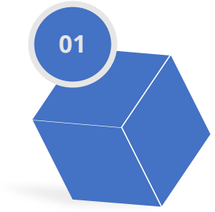
03 Logs and build status

04 Security and compliance

05 Pipeline cost

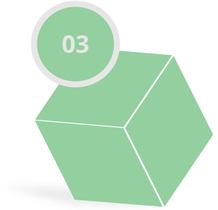
Pipeline Monitoring with CloudWatch

It is crucial for ensuring optimal health and functionality of the CI/CD processes. Following are the steps to set up CloudWatch events to monitor the CI/CD pipeline:

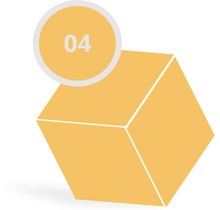


Integrate with EventBridge

Setup CloudWatch events



Send status notifications



Customize triggers with CloudWatch

Amazon EventBridge

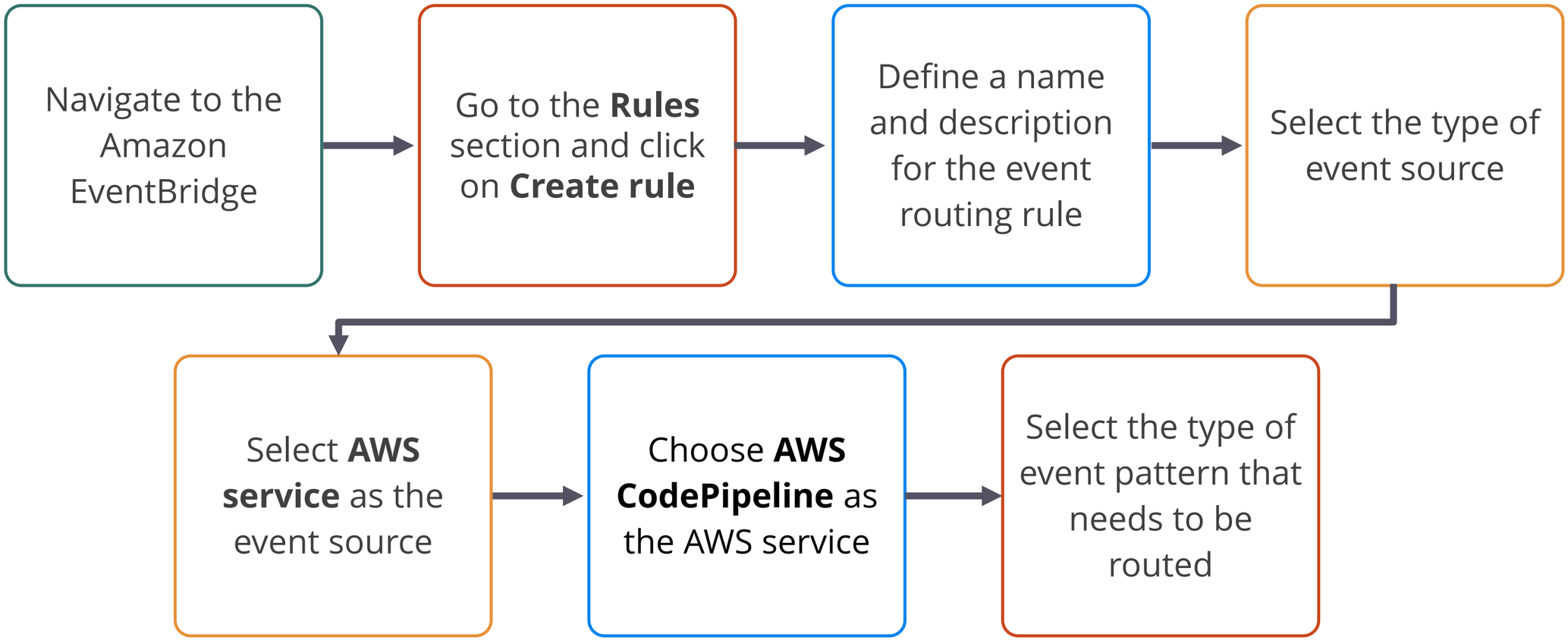
It is a serverless event router that simplifies the process of ingesting, filtering, and routing events from various sources to target destinations.

AWS EventBridge is used for:

- **Routing events** from diverse sources such as AWS services, custom applications, and third-party services
- **Defining rules** for event processing and routing
- **Integrating and managing events** from different AWS services

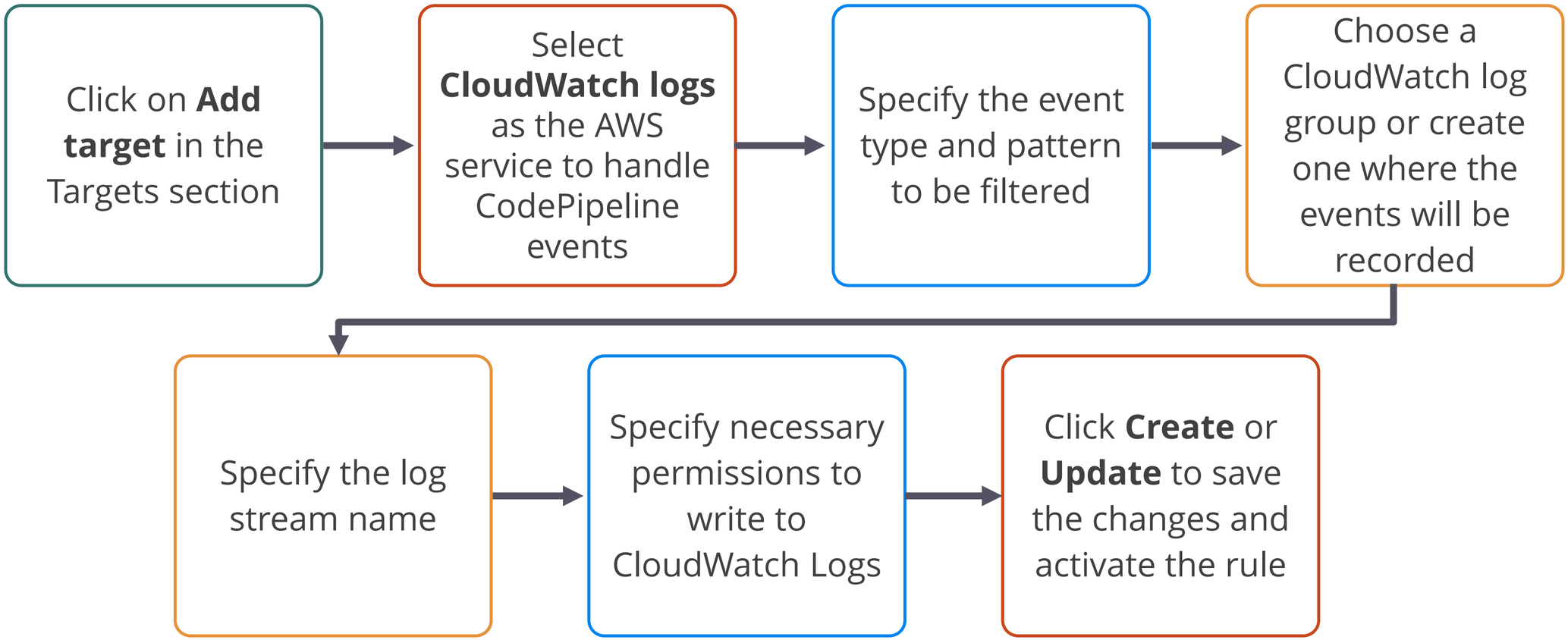
Integrating CI/CD Pipeline with EventBridge

Below are the steps for configuring EventBridge to route events from CodePipeline:



Setting up CloudWatch Events

After setting up CodePipeline as the event source, the following are the steps to set AWS CloudWatch as the target for Amazon EventBridge:



Setting up the Status Notification System

Below are the steps to integrate AWS Simple Notification Service (SNS) to set up a status notification system for monitoring CodePipeline events using CloudWatch:

1. Navigate to the AWS SNS console
2. Create a new topic where the notifications about CodePipeline will be sent
3. Add subscribers for the created topic

Setting up the Status Notification System

4 Choose the specific pipeline in the CodePipeline to monitor

5 Open the CloudWatch console, navigate to **Events**, and create a new rule

6 Choose **CodePipeline** and select the specific state changes or events to monitor

Setting up the Status Notification System

7 In the **Targets** section of the CloudWatch console, choose the created SNS topic

8 Configure the necessary permissions, allowing CloudWatch Events to publish the SNS topic

9 Save and activate the rule

Following these steps will result in notifications for specified CodePipeline events through the channels in the SNS topic.

Assisted Practice

Setting up CloudWatch Metrics and Alarms for a CI/CD Pipeline **Duration: 15 Min.**

Problem Statement:

You have been assigned a task to automate the continuous integration process as part of DevOps by implementing CloudWatch metrics and alarms on CodePipeline.

Outcome:

You can set up CloudWatch metrics and alarms for a CI/CD pipeline using AWS CodePipeline to automate the continuous integration process.

Note: Refer to the demo document for detailed steps:
05_Setting_up_CloudWatch_Metrics_and_Alarms_for_a_CI_CD_Pipeline

Assisted Practice: Guidelines

Steps to be followed:

1. Configure CloudWatch metrics and alarms on CodePipeline

ASSISTED PRACTICE

Assisted Practice

Integrating Infrastructure as Code into the CI/CD Pipeline **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to automate infrastructure deployment and enhance DevOps practices by integrating Terraform script execution into the AWS CodePipeline for efficient and consistent S3 bucket provisioning.

Outcome:

You can integrate Terraform script execution into AWS CodePipeline for automated infrastructure deployment and consistent S3 bucket provisioning.

Note: Refer to the demo document for detailed steps:
06_Integrating_Infrastructure_as_Code_into_the_CI_CD_Pipeline

Assisted Practice: Guidelines

Steps to be followed:

1. Configure the Git repository with the new Terraform template for deploying the S3 bucket
2. Configure CodePipeline to integrate Terraform script execution

ASSISTED PRACTICE

Quick Check

Your company is developing a microservices-based application where different services need to communicate via events. You need a solution to automatically route and process events from various AWS services, custom applications, and third-party services without managing any servers. Which AWS service would best fulfill this requirement?

A. It manages user authentication for AWS services.

B. It stores and retrieves data for AWS applications.

C. It provides real-time monitoring and alerting for AWS resources.

D. It serves as a serverless event router that handles event filtering, routing, and processing.

Key Takeaways

- AWS CodePipeline facilitates the creation of CI/CD pipelines, where it can automate the build and deployment phases.
- AWS CodeBuild is a fully managed build service that compiles source code, runs tests, and produces software packages within AWS.
- AWS Elastic Beanstalk is a Platform-as-a-Service (PaaS) in which developers deploy web applications, APIs, and microservices without delving into the underlying infrastructure details.
- AWS offers deployment strategies like Blue/Green, Canary, and Rolling to ensure seamless application updates with minimal downtime.

Implementing CI/CD for a Full Stack Application

Duration: **60 Minutes.**

Project Agenda: To establish a CI/CD pipeline for a full stack application deployment on AWS Elastic Beanstalk

Description:

This project guides you through the process of establishing a CI/CD pipeline on AWS for the streamlined deployment of a full-stack application. By the end of this guide, you will have a functional CI/CD pipeline that will ensure automated deployment for frontend and backend components.

LESSON-END PROJECT

Implementing CI/CD for a Full Stack Application

Duration: **60 Minutes.**

Perform the following :

1. Set up an Elastic Beanstalk backend application
2. Set up a Node.js Beanstalk environment for the backend application
3. Set up an Elastic Beanstalk frontend application
4. Set up a Node.js Beanstalk environment for the frontend application
5. Set up the CodePipeline for the backend application
6. Set up the CodePipeline for the frontend application
7. Deploy the application

Expected Deliverables: A functioning CI/CD pipeline on AWS for automated deployments

LESSON-END PROJECT

Thank you!